Схема архитектуры проекта RuTalk

Проект: Социальная сеть на Django

Состав документа

- Высокоуровневая схема слоёв и потока обработки запросов.
- Схема сущностей приложения и их связей.
- Краткое описание ролей основных компонентов проекта.
- Фиксация архитектурных правил, которые уже реализованы в коде.

1. Архитектурный контекст

RuTalk представляет собой монолитное веб-приложение на Django. В архиве присутствует один проектовый пакет social и одно прикладное приложение rutalk, в котором сосредоточены доменные модели, формы, маршруты, views, шаблоны и стили интерфейса.

Архитектурно проект построен по классической серверной MVC-подобной схеме Django: пользователь работает через браузер, URL-маршруты переводят запрос на function-based view, view обращается к формам и моделям, а затем формирует HTML-ответ через шаблон.

Ключевая архитектурная характеристика

Выделенного REST API, отдельного фронтенд-приложения и отдельного сервисного слоя в проекте нет. Приложение работает как единый Django-монолит с серверным рендерингом страниц.

2. Высокоуровневая схема проекта

На схеме ниже показаны основные слои проекта: клиентский уровень, веб-слой Django, доменная логика приложения, а также уровни хранения данных и медиа-файлов.

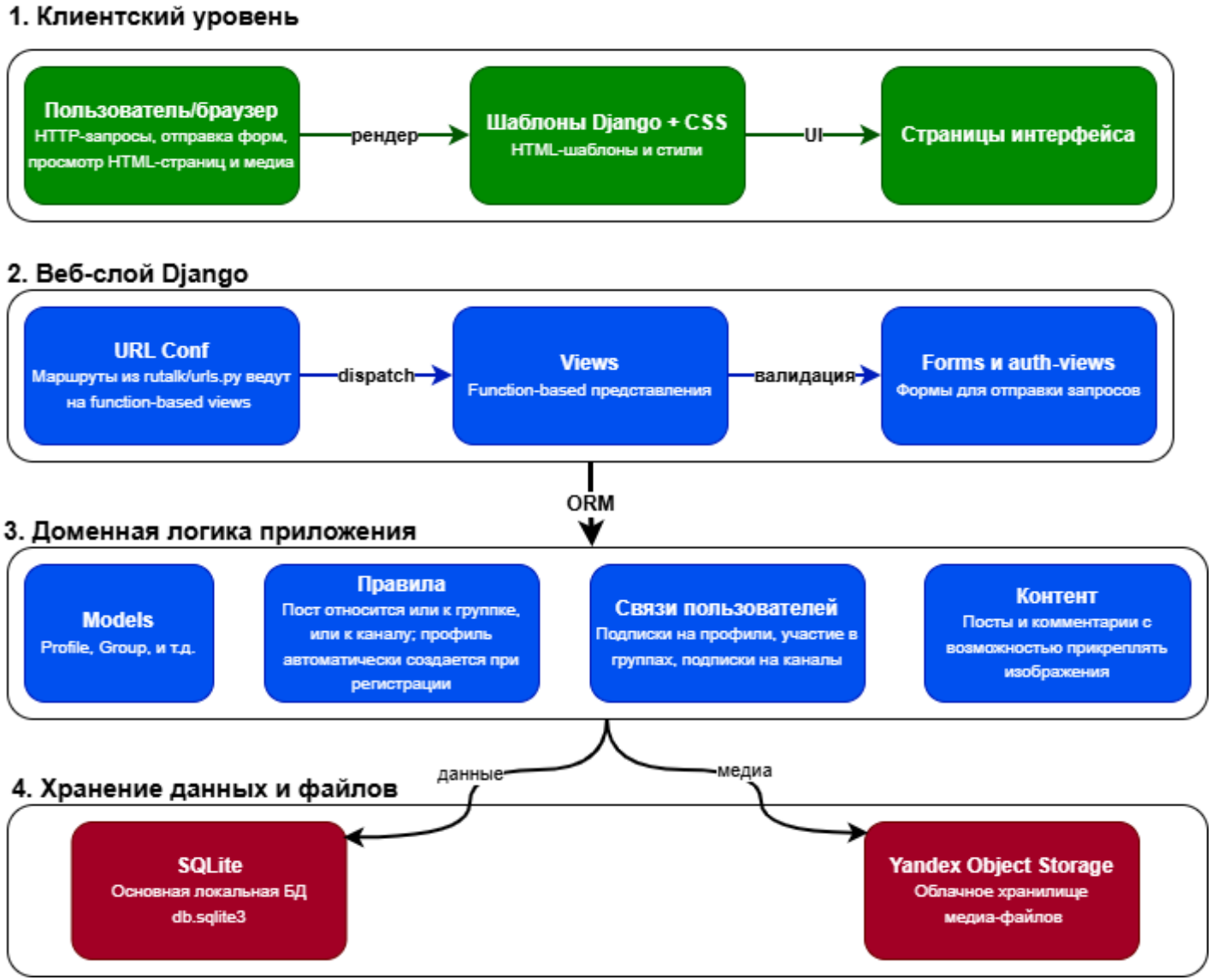

Рисунок 1. Высокоуровневая архитектура проекта RuTalk.

3. Как проходит запрос через систему

- Пользователь открывает страницу или отправляет форму из браузера.
- Маршрут из social/urls.py передаёт управление в rutalk/urls.py, где выбирается конкретная view-функция.
- View получает request, проверяет аутентификацию, при необходимости создаёт или валидирует форму.
- Для чтения и записи данных view использует ORM Django и модели приложения rutalk.
- После обработки запроса view возвращает HTML-страницу, собранную из шаблонов Django и CSS-стилей проекта.
- Изображения профилей, постов и комментариев выдаются либо из локального media-каталога, либо из Yandex Object Storage при активной S3-конфигурации.

4. Диаграмма сущностей и связей

Ниже приведена схема ключевых доменных сущностей. Она показывает, как в текущей реализации связаны пользователи, профили, группы, каналы, участия, посты и комментарии.

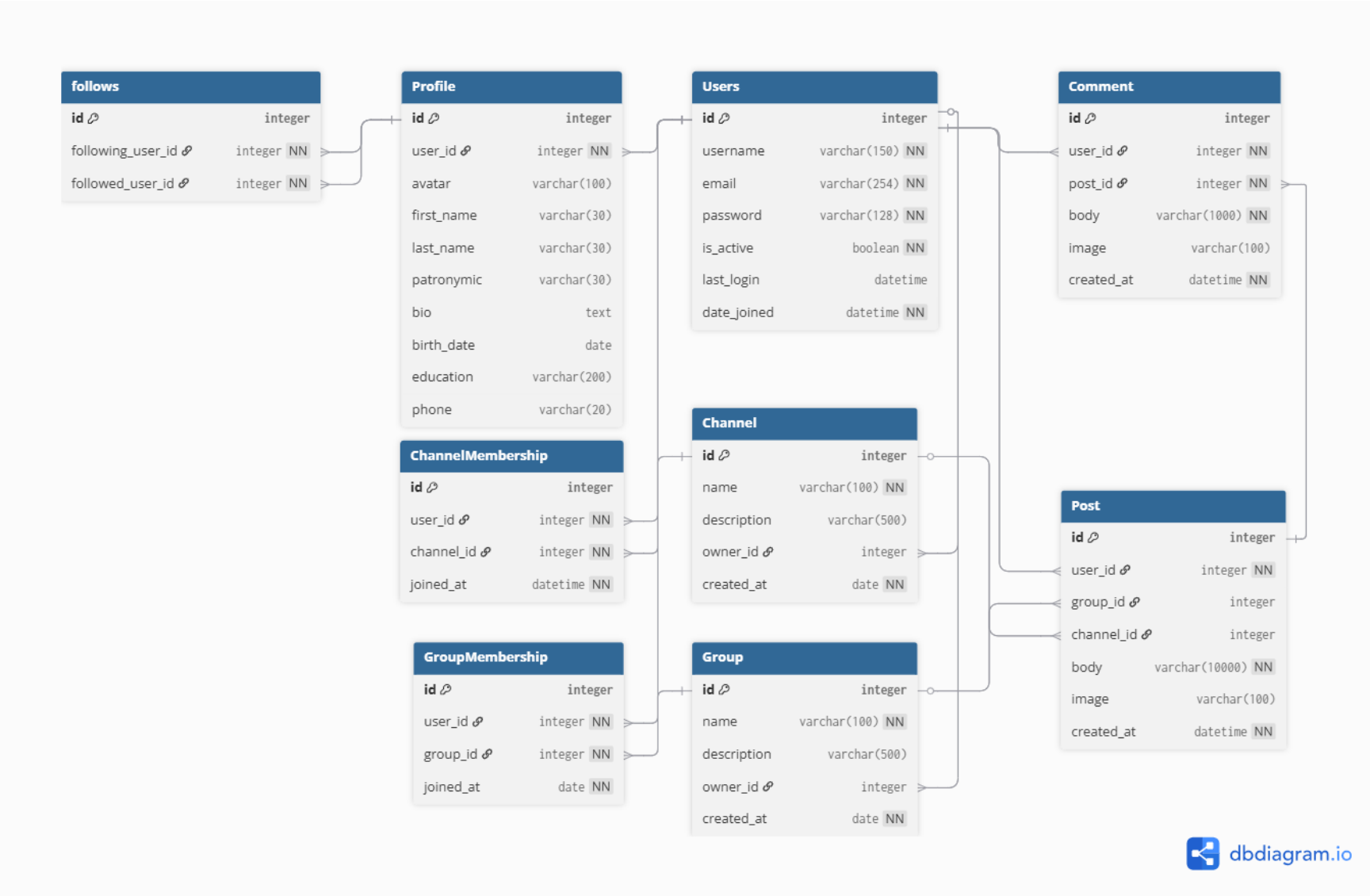

Рисунок 2. Основные модели приложения и их связи.

5. Роли основных компонентов

Компонент	Назначение
manage.py	Точка входа для административных команд Django.
social/settings.py	Общие настройки проекта: БД, шаблоны, media, auth, storages, debug-режим.
social/urls.py	Корневой URLConf, подключающий маршруты rutalk и admin.
rutalk/models.py	Доменные сущности социальной сети и правила их сохранения.
rutalk/forms.py	ModelForm и кастомные auth-формы для пользовательских сценариев.
rutalk/views.py	Основная прикладная логика страниц и действий пользователя.
rutalk/templates/	HTML-шаблоны интерфейса и страниц авторизации.
rutalk/static/rutalk/css/app.css	Единый файл стилей пользовательского интерфейса.
db.sqlite3	Локальная база данных разработки.
.env	Переменные окружения: секретный ключ Django и параметры Yandex Object Storage.

6. Архитектурные правила, зафиксированные в коде

- При создании пользователя автоматически создаётся запись Profile через сигнал post_save.
- Новый профиль сразу подписывается на самого себя: user_profile.follows.add(instance.profile).
- Сущность Post должна принадлежать ровно одному контейнеру: либо Group, либо Channel. Это контролируется в переопределённом методе save().
- Публиковать сообщения в группе может участник группы; в канале форма создания поста показывается только владельцу канала.
- Для Membership и ChannelMembership введены ограничения unique_together, запрещающие дублировать участие одного и того же пользователя.